

Final Project

Group: Jacob Lebovitz, Brett Matthews, Santiago Ramirez

May 5, 2025

Data Selection

Choice of Dataset: We selected the Yelp Academic Dataset, a collection of user reviews, business data, and user accounts, ideal for studying business performance and user-user interactions with them on the site. This dataset meets project requirements with its ~1GB size (following sampling), relational structure (businesses, reviews, users), and JSON Lines (JSONL) format, enabling relational (PostgreSQL) and non-relational database exploration.

Concessions and Sampling: In order to handle computational limitations on the local macOS environment, the dataset was sampled down to approximately 50,000 users, approximately 50,000 businesses, and approximately 1,000,000 reviews using a Python script (sample_data.py). Sampling was done randomly to maintain data distribution, maintaining population representativeness within local resource capacity. The whole sampling script is on our computer but is too long to show.

Systems and Database Setup

Compute and Storage: We store and compute on a local macOS machine using PostgreSQL 14. This is in an effort to circumvent DataHub's limits on our approximately 1GB dataset with complete environmental control. There are no cloud platforms.

Loading Data into PostgreSQL and MongoDB: We imported sample_user.json, sample_business.json, and sample_review.json from the local folders to the yelp_project database via the Python script, inserting the values one by one in batches, and in case of failure, simply continued.

Code snippet:

```
Unset
#for mongoDB
if review.count_documents({}) == 0:
    with open(REVIEW_PATH, encoding='utf-8') as f:
        for line in f:
            if review.count_documents({}) == 1000000:
                break
            try:
                obj = json.loads(line)
                review.insert_one(obj)
            except json.JSONDecodeError:
                continue

#for SQL
conn = psycopg2.connect(**DB_PARAMS)
```

```

cur = conn.cursor()

with open(SRC, 'r') as fin:
    for chunk in fin:
        buffer += chunk
        try:
            # raw_decode returns (obj, end_index)
            while buffer:
                obj, idx = decoder.raw_decode(buffer)
                buffer = buffer[idx:].lstrip()
                batch.append((
                    obj.get('user_id'),
                ))

            good += 1
            if len(batch) >= BATCH_SIZE:
                execute_batch(cur,
                    """
                    INSERT INTO dataset()
                    """,
                    batch
                )
                conn.commit()
                batch.clear()

```

Tool Comparison

PostgreSQL outperforms MongoDB for structured data queries, such as listing companies by state (354 ms vs. 46,279 ms), due to effective indexing and window functions. MongoDB's performance suffers from full-collection scans without indexes. PostgreSQL is ideal for consistent, report-driven applications with complex joins but lacks schema flexibility. MongoDB excels with unstructured, rapidly changing data and horizontal scaling, though it requires careful index planning for analytics. Choose PostgreSQL for structured, query-intensive tasks; MongoDB for scalable, dynamic datasets.

Queries:

- **Query 1: Top Businesses by State**
 - SQL:

```

yelp_project=# WITH RankedBusinesses AS (
    SELECT
        b.name,
        b.city,
        b.state,
        b.stars,
        b.review_count,
        b.latitude,
        b.longitude,
        RANK() OVER (PARTITION BY b.state ORDER BY b.stars DESC, b.review_count DESC) AS state_rank,
        AVG(b.stars) OVER (PARTITION BY b.state) AS state_avg_stars,
        COUNT(*) OVER (PARTITION BY b.city) AS city_business_count,
        SQRT(
            POW(b.latitude - AVG(b.latitude) OVER (PARTITION BY b.state), 2) +
            POW(b.longitude - AVG(b.longitude) OVER (PARTITION BY b.state), 2)
        ) AS distance_from_state_center
    FROM business b
    WHERE
        b.review_count >= 10
        AND b.stars IS NOT NULL
)
SELECT
    name,
    city,
    state,
    stars,
    review_count,
    state_rank,
    state_avg_stars,
    city_business_count,
    distance_from_state_center,
    CASE
        WHEN stars > state_avg_stars THEN 'Above Average'
        WHEN stars = state_avg_stars THEN 'Average'
        ELSE 'Below Average'
    END AS competitive_position
FROM RankedBusinesses
WHERE state_rank <= 3
ORDER BY state, state_rank
LIMIT 15;

```

○ MongoDB:

```

q1_cursor = business.aggregate([
    { "$match": { "review_count": { "$gte": 10 }, "stars": { "$ne": None } } },
    { "$group": {
        "_id": "$state",
        "avgStars": { "$avg": "$stars" },
        "avgLat": { "$avg": "$latitude" },
        "avgLon": { "$avg": "$longitude" },
        "data": { "$push": {
            "name": "$name",
            "city": "$city", "state": "$state", "stars": "$stars",
            "review_count": "$review_count", "latitude": "$latitude",
            "longitude": "$longitude"
        } }
    } },
    { "$unwind": "$data",
    { "$group": {
        "_id": { "st": "$data.state", "ct": "$data.city" },
        "avgStars": { "$first": "$avgStars" },
        "avgLat": { "$first": "$avgLat" },
        "avgLon": { "$first": "$avgLon" },
        "cityCnt": { "$sum": 1 },
        "allData": { "$push": {
            "name": "$data.name", "city": "$data.city", "state": "$data.state",
            "stars": "$data.stars", "review_count": "$data.review_count",
            "latitude": "$data.latitude", "longitude": "$data.longitude",
            "dist": {
                "$sqrt": {
                    "$add": [
                        { "$pow": [ { "$subtract": ["$data.latitude", "$avgLat"] }, 2 ] },
                        { "$pow": [ { "$subtract": ["$data.longitude", "$avgLon"] }, 2 ] }
                    ]
                }
            }
        } }
    } }
    } ],
    { "$sort": { "state": 1, "state_rank": 1 } },
    { "$limit": 15 }
])

```

```

{ "$unwind": "$allData",
  { "$sort": { "_id.st": 1, "allData.stars": -1, "allData.review_count": -1 } },
  { "$group": {
    "_id": { "state": "$_id.st" },
    "avgStars": { "$first": "$avgStars" },
    "tmp": { "$push": {
      "n": "$allData.name", "c": "$allData.city", "s": "$allData.state",
      "st": "$allData.stars", "rc": "$allData.review_count", "d": "$allData.dist",
      "cc": "$cityCnt", "r": { "$sum": 1 }
    } }
  } },
  { "$unwind": "$tmp",
    { "$match": { "tmp.r": { "$lte": 3 } } },
    { "$sort": { "_id.state": 1 } },
    { "$group": {
      "_id": "$_id.state",
      "avgStars": { "$first": "$avgStars" },
      "tmp": { "$push": {
        "n": "$tmp.n", "c": "$tmp.c", "s": "$tmp.s", "st": "$tmp.st",
        "rc": "$tmp.rc", "d": "$tmp.d", "cc": "$tmp.cc", "r": "$tmp.r"
      } }
    } },
    { "$unwind": "$tmp",
      { "$project": {
        "_id": 0, "name": "$tmp.n", "city": "$tmp.c", "state": "$tmp.s",
        "stars": "$tmp.st", "review_count": "$tmp.rc", "state_rank": "$tmp.r",
        "state_avg_stars": "$avgStars", "city_business_count": "$tmp.cc",
        "distance_from_state_center": "$tmp.d",
        "competitive_position": {
          "$cond": {
            "if": { "$gt": ["$tmp.st", "$avgStars"] }, "then": "Above Average", "else": {
              "$cond": {
                "if": { "$eq": ["$tmp.st", "$avgStars"] },
                "then": "Average",
                "else": "Below Average"
              }
            }
          }
        }
      } }
    } },
    { "$sort": { "state": 1, "state_rank": 1 } },
    { "$limit": 15 }
  ]
)

```

Task: The query filters for businesses that have a minimum of 10 reviews, returns businesses ranked by stars and review_count within state, calculates state-average stars and city business counts, and returns the top 3 businesses per state (up to 15 results), along with a competitive position determined by state-average stars.

Relevance: This workload is well-suited to benchmarking PostgreSQL and MongoDB because it is targeting advanced analytical features, including window functions (Postgres) and multi-step aggregation

pipelines (MongoDB). It tests their performance at grouping, ranking, and filtering and is an appropriate benchmark for analytical query performance.

PostgreSQL is significantly quicker than MongoDB (354.39 ms compared to 46,279 ms) through window function optimization and query planning despite disk-based sorting (6440kB). The full-document scan (COLLSCAN) of 150,346 documents and many \$group/\$unwind stages in the MongoDB pipeline take time. Indexing on businesses.review_count and businesses.stars reduces the PostgreSQL time to ~300 ms and the MongoDB time to ~10,000 ms, but the complexity of the MongoDB pipeline is still the main hindrance. PostgreSQL handles well-structured, analytic queries but MongoDB is designed for flexible, high-write workloads. This is high-read.

- **Query 2: Top Reviewers**

- SQL:

```
Unset
EXPLAIN ANALYZE
WITH base_users AS (
  SELECT user_id, name
  FROM   users
  LIMIT  10
)
SELECT
  bu.user_id,
  bu.name,
  COUNT(r.review_id) AS review_count
FROM   base_users AS bu
LEFT   JOIN reviews r
      ON r.user_id = bu.user_id
GROUP  BY bu.user_id, bu.name
ORDER  BY review_count DESC;
```

user_id	name	review_count
j14WgRoU_-2ZE1aw1dXrJg	Daniel	12
q_QQ5kBBwLCcbL1s4NVK3g	Jane	5
SZDeASXq7o05mMNLshsdIA	Gwen	5
AUi8MPWJ0mLkMfwbui27lg	John	4
2WnXYQFK0hXEoTxPtV2zvq	Steph	3
cxuxXkcihfCbqt5Byrup8Q	Rob	3
qVc8ODYU5SZjKXVBgXdI7w	Walker	3
E9kcwJdJUHUTKfQurPljwA	Mike	2
hA5lMy-EnncsH4JoR-hFGQ	Karen	2
l01iq-f75hnPNZkTy3Zerg	Rachelle	1

- MongoDB:

```
Unset
user.aggregate([
  {
    "$limit": 10
  },
  {
    "$lookup": {
      "from": "review",
      "localField": "user_id",
      "foreignField": "user_id",
      "as": "user_reviews"
    }
  }
])
```

```
[{'_id': ObjectId('6818071db7e908597926285d'),
  'user_id': 'j14WgRoU_-2ZE1aw1dXrJg',
  'name': 'Daniel',
  'review_count': 12},
{'_id': ObjectId('6818071db7e908597926285f'),
  'user_id': 'SZDeASXq7o05mMNLshsdIA',
  'name': 'Gwen',
  'review_count': 5},
{'_id': ObjectId('6818071db7e9085979262861'),
  'user_id': 'q_QQ5kBBwLCcbL1s4NVK3g',
  'name': 'Jane',
  'review_count': 5},
{'_id': ObjectId('6818071db7e9085979262865'),
  'user_id': 'AUi8MPWJ0mLkMfwbui27lg',
  'name': 'John',
  'review_count': 4},
{'_id': ObjectId('6818071db7e908597926285c'),
  'user_id': 'qVc8ODYU5SZjKXVBgXdI7w',
  'name': 'Walker',
  'review_count': 3},
{'_id': ObjectId('6818071db7e908597926285e'),
  'user_id': 'E9kcwJdJUHUTKfQurPljwA',
  'name': 'Mike',
  'review_count': 2},
{'_id': ObjectId('6818071db7e908597926285b'),
  'user_id': 'hA5lMy-EnncsH4JoR-hFGQ',
  'name': 'Karen',
  'review_count': 2},
{'_id': ObjectId('6818071db7e908597926285a'),
  'user_id': 'l01iq-f75hnPNZkTy3Zerg',
  'name': 'Rachelle',
  'review_count': 1}]
```

```

    },
    {
      "$project": {
        "user_id": 1,
        "name": 1,
        "review_count": { "$size": "$user_reviews" }
      }
    },
    {
      "$sort": { "review_count": -1 }
    }
  ]
)

```

Task: The query retrieves the top 10 users' most active reviewers.

This is accomplished by a join of the review collections/tables and the users, grouping by user_id and name, counting reviews, and ordering in descending order by count.

Relevance: This is a suitable task for PostgreSQL and MongoDB to benchmark, since it exercises their join (or lookup), aggregation, and ordering features—essential for user behavior analysis. This brings out PostgreSQL's SQL joins power and MongoDB's flexibility through aggregation pipelines as a good bench on which to benchmark performance and optimization.

Comparison: Based on the execution plans you've tried, MongoDB comes nearly even with PostgreSQL = 3 162 ms to 3 355 ms. MongoDB materializes the user collection down to 10 documents before the \$lookup, so at most 10 index-free probes query the 1 M-row review collection versus PostgreSQL accessing all 1 M reviews sequentially and hash-joining with 50 k users before grouping and sorting. With otherwise humble user_id index on both sides, both engines are sub-second (typically ≈ 500–600 ms), so performance adequate for workload is the only consideration: PostgreSQL is excellent at many-relational analytics and SQL feature-rich, and MongoDB shines when your data are flexible-schema and you max out document-oriented aggregation. pipelines.

- **Query 3:**

- SQL:

Unset

```

SQL: SELECT user_id, COUNT(*) AS
      total_reviews
FROM review GROUP BY user_id
ORDER BY total_reviews
DESC LIMIT 20;

```

user_id	total_reviews
_BcWyKQL16ndpBdggh2kNA	483
Xw7ZjaGfr0wNVt6s_5KZfA	283
0Igx-a1wAstiBDerGxXk2A	264
bYENop4BuQepBjM1-BI3fA	240
-G7Zkl1wIwBBmD0KRy_sCw	224
ou0DopBKF3AqfCkuQEnrDg	210
1HM81n6n4iPIFU5d2Lokhw	209
Um5bfs5DH6eizgjH3xZsvg	208
qj fMBIZpQT9DDtw_BwCopQ	203

- MongoDB:

Unset

```
{
  "$group": {
    "_id": "$user_id",
    "total_reviews": {"$sum": 1}
  },
  {"$sort": {"total_reviews": -1}},
  {"$limit": 20},
  {
    "$project": {
      "_id": 0,
      "user_id": "$_id",
      "total_reviews": 1
    }
  }
}
```

```
{'total_reviews': 483, 'user_id': '_BcWyKQL16ndpBdggh2kNA'},
{'total_reviews': 283, 'user_id': 'Xw7ZjaGfr0WNVt6s_5KZfA'},
{'total_reviews': 264, 'user_id': '0Igx-alwAstiBDerGxXk2A'},
{'total_reviews': 240, 'user_id': 'bYENop4BuQepBjM1-BI3fA'},
{'total_reviews': 224, 'user_id': '-G7Zkl1wIWBbmDOKRy_sCW'},
{'total_reviews': 210, 'user_id': 'ouODopBKF3AqfCkuQEmrDg'},
{'total_reviews': 209, 'user_id': 'IHM8In6n4iPIFU5d2Lokhw'},
{'total_reviews': 208, 'user_id': 'Um5bfs5DH6eizgjH3xZsvg'},
{'total_reviews': 203, 'user_id': 'qjIMBIZpQT9DDtw_BWCopQ'},
{'total_reviews': 199, 'user_id': 'ET8n-r7glWYqZhuR6GcdNw'},
{'total_reviews': 193, 'user_id': 'fr1Hz2acAb3OaL3l6DyKNg'},
{'total_reviews': 181, 'user_id': 'wXdbkFZsIDR7uJvbWElyA'},
{'total_reviews': 173, 'user_id': 'eTvp_hYnsrI5-ow_sQ31_g'},
{'total_reviews': 173, 'user_id': 'I2XpWCHAom1JRyHXZQmfg'},
{'total_reviews': 162, 'user_id': 'IAW03ccfMEuRZiDqvVgc0w'},
{'total_reviews': 161, 'user_id': 'CfX4sTIFFNaRchnsqwVfg'},
{'total_reviews': 160, 'user_id': 'TYQk0R6sPfo3WeX-1_5BuA'},
{'total_reviews': 158, 'user_id': 'B-s-8VUnuBjGTP3d01jsyw'},
{'total_reviews': 157, 'user_id': '2iS1vg5TYpV_iEiNC8osTg'},
{'total_reviews': 157, 'user_id': 'PnwOegg7RXfMeNAyO9fQhQ'}}
```

Task: The query presents the 20 most active reviewers, which was ranked by the number of reviews each unique user_id completed, sorted by descending order of review count.

Relevance: This is an appropriate task for testing PostgreSQL and MongoDB because it exercises their ability to group, aggregate, and sort. It shows how well PostgreSQL can work with structured data using SQL aggregates and how MongoDB can quickly work with aggregation pipelines, so it is a good benchmark for evaluating query performance and optimization strategies.

Comparison: PostgreSQL outperforms MongoDB on this query due to its optimized hash aggregate and lower overhead, even in the absence of an index (918 ms vs. 12,360 ms). Both benefit immensely from indexing, but the pipeline complexity of MongoDB renders it ineffective for simple aggregations. PostgreSQL is the way to go for structured, query-heavy work, and MongoDB is well-suited for flexible, high-write workloads with proper indexing.

- **Query 4:**

- SQL:

Unset

```
SELECT review_count, AVG(average_stars) AS avg_stars_per_reviews
FROM users
GROUP BY review_count
ORDER BY avg_stars_per_reviews DESC;
```

review_count	avg_stars_per_reviews
1600	4.8500000000000000
1449	4.7100000000000000
1527	4.6000000000000000
1056	4.4800000000000000
1301	4.4800000000000000
1896	4.4700000000000000
1457	4.4300000000000000
3004	4.4300000000000000
1586	4.4200000000000000

- MongoDB:

Unset

```
query6 = [
  { "$group": {"_id": "$review_count", "avg_stars_reviews": {"$avg": "$average_stars"}}
},
```

```
{'_id': 1600, 'avg_stars_per_reviews': 4.85},
{'_id': 1449, 'avg_stars_per_reviews': 4.71},
{'_id': 1527, 'avg_stars_per_reviews': 4.6},
{'_id': 1301, 'avg_stars_per_reviews': 4.48},
{'_id': 1056, 'avg_stars_per_reviews': 4.48},
{'_id': 1896, 'avg_stars_per_reviews': 4.47},
{'_id': 1457, 'avg_stars_per_reviews': 4.43},
{'_id': 3004, 'avg_stars_per_reviews': 4.43},
{'_id': 1586, 'avg_stars_per_reviews': 4.42},
{'_id': 3012, 'avg_stars_per_reviews': 4.41},
{'_id': 3683, 'avg_stars_per_reviews': 4.4},
{'_id': 1015, 'avg_stars_per_reviews': 4.4},
{'_id': 1882, 'avg_stars_per_reviews': 4.39},
{'_id': 1506, 'avg_stars_per_reviews': 4.39},
{'_id': 1700, 'avg_stars_per_reviews': 4.37},
{'_id': 2481, 'avg_stars_per_reviews': 4.37},
{'_id': 1391, 'avg_stars_per_reviews': 4.37},
```

```
{ "$sort":{"avg_stars_reviews": -1}
}]
results = list(user.aggregate(query6))
```

Task: This query explores the connection between the number of reviews a user has given and the average number of stars users with that exact review count tend to give on average. This is done through grouping on review_count and then aggregating an average over the average_stars.

Relevance: This query is useful for testing the differences between our SQL system MongoDB because it is an example of a group function on a feature that has many different distinct values. This will test the limits of the grouping function, and make any differences very clear in their efficiency and fitness.

Performance Analysis: PostgreSQL executed the query in 2,487.47 ms, with a parallel sequential scan on 1M rows and a hash aggregate (833kB memory); a sequential scan was the bottleneck. Adding an index on users.review_count (CREATE INDEX idx_users_review_count ON users (review_count);) reduced the time to ~1,800 ms by enabling an index scan, though sorting was still costly. MongoDB executed the pipeline in 72 ms, scanning 42,326 documents (COLLSCAN). Indexing user.review_count (user.create_index([["review_count", 1]])) brought minor improvement (70 ms), as grouping dominated. Row filtering also could optimize them further.

Comparison: MongoDB fares better than PostgreSQL (72 ms vs. 2,487 ms) since it uses a smaller dataset and efficient in-memory aggregation. PostgreSQL is well with large tables but slower without selective filtering. Both are helped by indexing, but MongoDB's pipeline works more efficiently for that purpose.

- **Query 5:**
 - SQL:

```
Unset
SELECT COUNT(*)
FROM review AS b
WHERE review_text LIKE '%best%';
```

- MongoDB:

```
Unset
query4 = [
  { "$match": { "$text": { "$search": "best" } }
}
]
len(list(review.aggregate(query4)))
```

Task: This query looks at the number of reviews that mention the word “best.” It examines the text attribute in each review to see if it contains the keyword “best.” This could be useful in an analysis of review bias and positivity.

Relevance: This is a helpful question in the analysis of our database systems because it highlights the system's ability to search in a non-indexed field of a dataset. The search function, through WHERE and \$match respectively, are valuable functions in SQL and MongoDB. Therefore, we felt it was important to see how the databases responded to a large search on a field that was almost entirely unique to each review.

Performance Analysis: The execution time of the above query on PostgreSQL was 3138 ms and the execution time of the query on MongoDB was 3669 ms. Although SQL is supposed to run faster with rectangular, known data sets because of its characteristic of optimizing the performance of the query, Mongo and SQL difference is not a large one. This is because the above filtering query cannot be optimized by any form of indexing or aggregating. But since there was a text field as an attribute in each review, it was slightly faster for SQL to deal with the uniform pattern of filtering.

Comparison: SQL's ability to semi-aggregate the data makes it slightly more run time efficient than MongoDB. The parallel sequential scan in SQL over the data with 2 workers also increases the efficiency of the operation. Therefore, with semi-rectangular data found in reviews, it would be ideal to employ a SQL based data management system if data filtering is your main goal.

Individual Reflections:

Santiago: Our benchmarks demonstrated PostgreSQL reliably gave the best join performance, GROUP BY queries, and searching. My only hangup is data insertion friction: coding at the terminal feels clumsy at times, and the guides complicate adjusting columns, values, and associations beyond necessity. MongoDB is a smoother and friendlier process in all of these aspects, albeit one in which the query processor hasn't caught up with the comprehensive speed of PostgreSQL.

Brett:

Jacob: This project was a very informative insight into the differences in database management systems. I found it very helpful to understand the capacities of different systems and how they interpret different queries. This project will give me some more tools to understand the thought that goes into choosing systems. Similar to Santiago, I struggle with data insertion and the challenges with uploading data into unfamiliar software.